

TERS-ABNet Tutorial

Practical guide for inference, result reproduction, and model scope

This tutorial helps new researchers and users understand TERS-ABNet, run the supplied examples, and reproduce the representative results reported in the manuscript. In this document, <repo> denotes the `code_upstall_new` repository root.

1. Overview and Background

TERS-ABNet addresses a high-dimensional inverse problem: reconstructing an explicit molecular atom-bond graph from single-molecule tip-enhanced Raman spectroscopy (TERS) mappings. A single vibrational-mode image is not uniquely attributable to one atom or bond, because its signal can contain overlapping and interfering contributions from neighboring atoms and chemical bonds. TERS-ABNet therefore integrates spatially resolved information from many spectral windows and converts it into a structured molecular representation.

The framework has two prediction tracks:

- **ANet** predicts element-resolved atom probability maps for C, N, O, and H.
- **BNet** predicts bond or functional-group probability maps. The planar model has 9 output channels; the transfer-learning model has 11 output channels, adding methyl and ethyl groups.
- **Post-processing** detects Gaussian-like peaks in the probability maps, assigns their positions and types, then combines distance, bond/group, and valence constraints to construct a molecular graph and write an SDF structure.

The manuscript uses $128 \times 128 \times 160$ TERS mappings spanning 2.5×2.5 nm². The 160 spectral channels are constructed from 20 cm⁻¹ wavenumber windows, and the atom/bond labels are Gaussian spots centered on the reference positions. The reported independent-test results include an atomic coordinate MAE of about 0.233 Å, a bond-position MAE of about 0.199 Å, mean atom-type accuracy of 94.2%, and mean bond-type accuracy of 92.0%.

Connection between repository results and manuscript figures

Manuscript result	Example directory	Script	Main outputs to inspect
Fig. 1 workflow and Supplementary Fig. 3 (2,6-dihydroxypyridine)	<code>examples/1_plane_mole/</code>	<code>test_ABNet_plane_mole.sh</code>	Input TERS montage, atom/bond maps, coordinates, reconstructed geometry
Fig. 3 and Supplementary Fig. 4 (four planar molecules)	<code>examples/3-1_plane_mole/</code> <code>through 3-4_plane_mole/</code>	<code>test_ABNet_plane_mole.sh</code>	Probability maps and <code>geometry.sdf</code>

Manuscript result	Example directory	Script	Main outputs to inspect
Fig. 4 resolution-gene realization examples	examples/4-1_plane_mole/ through 4-3_plane_mole/	test_ABNet_plane_mole.sh	Atom and bond probability maps at three input conditions
Fig. 5 and Supplementary Fig. 6 (four nonplanar molecules)	examples/5-1_nonplane_mole/ through 5-4_nonplane_mole/	test_ABNet_nonplane_mole.sh	11-channel BNet outputs, coordinates, and geometry
Fig. 6 simulated and experimental MgP	examples/6-1_MgP_sim/ 6-2_MgP_exp/	test_ABNet_plane_MgP.sh	Atom and bond probability maps

docs/Maunscript-20260316-arxiv.docx contains the manuscript draft and docs/SI-TERS-ABNet-20260316-clean.docx contains the supplementary information. The table above maps code outputs to figure content; the scripts generate result files but do not automatically compose publication-ready figure panels.

2. Repository Architecture

```
<repo>/
■■■■ checkpoints/           # Four pretrained model checkpoints
■   ■■■■ ANet_base/         # Atom network for planar molecules
■   ■■■■ BNet_base/         # 9-channel bond network for planar molecules
■   ■■■■ ANet_transfer/     # Atom network after transfer learning
■   ■■■■ BNet_transfer/     # 11-channel bond network after transfer learning
■■■■ examples/             # Representative inputs and regenerable results
■■■■ scripts/               # Bash entry points for training and inference
■■■■ src/
■   ■■■■ models/attention_unet.py # Attention U-Net architecture
■   ■■■■ models/attention_gate.py # Attention gates for skip connections
■   ■■■■ test_ABNet_base.py       # Planar inference, peak detection, graph construction
■   ■■■■ test_ABNet_transfer.py   # Nonplanar inference, peak detection, graph construction
■   ■■■■ test_ABNet_base_MgPsim.py # Simulated MgP inference
■   ■■■■ test_ABNet_base_MgPexp.py # Experimental MgP inference
■   ■■■■ train_ANet_base.py       # Base ANet training
■   ■■■■ train_BNet_base.py       # Base BNet training
■   ■■■■ train_ANet_transfer.py   # ANet transfer learning
■   ■■■■ train_BNet_transfer.py   # BNet transfer learning
■   ■■■■ c/                      # C++/Python bindings for template matching
■■■■ docs/                    # Manuscript and supplementary information
■■■■ requirements.txt          # Pinned Python dependencies
■■■■ README.md                 # Original project summary
```

Example-directory files

Each example is driven by ters_map.pt. In directories processed with - -geom, the inference program writes or overwrites the following artifacts.

File	Description
ters_map.tif	Montage of 160 input TERS maps, each shown with its own scale
ters_map_same_scale.tif	The same montage shown on a common 0-1 scale
atom_map.tif	Merged element-resolved probability maps from ANet
bond_map.tif	Merged bond/group probability maps from BNet
atom_coord.txt	Peak-detected atomic coordinates and labels
bond_coord.txt	Peak-detected bond/group-center coordinates and labels

File	Description
atom_coord.svg	Vector plot of predicted atom positions
bond_coord.svg	Vector rendering of inferred connectivity
geometry.sdf	V3000 SDF molecular graph selected during post-processing
args.pkl	Serialized command-line arguments for the run

The 4- * planar-resolution cases and the MgP scripts do not enable - -geom. Their intended outputs are atom and bond probability maps, not automatically generated coordinates, SVGs, or SDF geometries.

3. Environment and Quick Start

Requirements

Python 3.8 is recommended. The repository pins PyTorch 1.12.0, NumPy 1.21.5, SciPy 1.7.3, scikit-image 0.19.2, Matplotlib 3.5.1, Pandas 1.4.2, scikit-learn 1.0.2, tqdm 4.64.0, and pybind11 2.10.1. CUDA is recommended for training and large-scale inference. The supplied inference code does not explicitly move models or tensors to CUDA, so test a small example in the target environment first.

On Linux, WSL, or Git Bash:

```
cd <repo>
python -m venv .venv
source .venv/bin/activate          # PowerShell: .venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If PyTorch must be installed from a platform-specific CPU or CUDA wheel, install mutually compatible torch==1.12.0, torchvision==0.13.0, and torchaudio==0.12.0 first, then install the remaining dependencies.

Quick start: planar inference

The Bash scripts use relative paths such as ../src and ../checkpoints. Enter the scripts/ directory before running them:

```
cd <repo>/scripts
bash test_ABNet_plane_mole.sh
```

After completion, inspect the newest atom_map.tif, bond_map.tif, atom_coord.txt, bond_coord.txt, and geometry.sdf in ../examples/1_plane_mole/. This is the shortest complete workflow: TERS tensor -> ANet/BNet -> probability maps -> peak positions -> molecular graph.

On Windows PowerShell with a usable Git Bash or WSL bash command:

```
Push-Location '<repo>\scripts'
bash .\test_ABNet_plane_mole.sh
Pop-Location
```

Do not run `bash scripts/test_ABNet_plane_mole.sh` from the repository root. In that case, the paths inside the script resolve one directory above the repository and the program will not find `src` or checkpoints.

4. Core Workflow and Reproducing Results

Step 0: Select the correct model and input representation

1. Use `ANet_base` and `BNet_base` for planar molecules. Use both `*_transfer` checkpoints for controlled nonplanar cases containing methyl/ethyl groups. Use the dedicated `MgP` scripts for the `MgP` examples.
2. Confirm that `ters_map.pt` is a loadable PyTorch tensor with a batch dimension and is compatible with the trained representation: 160 spectral channels on a 128×128 grid.
3. Do not mix `BNet` checkpoint families and output dimensions: base `BNet` requires `--out_channels_b 9`; transfer `BNet` requires `--out_channels_b 11`. Both `ANet` models use 4 channels.
4. For new data, create a separate output directory and set `--savedir` accordingly. This preserves the reference artifacts in `examples/`.

Step 1: Reproduce planar-molecule results (Fig. 1, Fig. 3, Fig. 4)

```
cd <repo>/scripts
bash test_ABNet_plane_mole.sh
```

The script processes `1_plane_mole`, `3-1` through `3-4`, and `4-1` through `4-3`.

- `1_plane_mole` is the 2,6-dihydropyridine example used to explain the Fig. 1 workflow. Compare its complete output chain with Supplementary Fig. 3.
- `3-1` through `3-4` correspond to the short-chain, six-membered heterocycle, fused heterocycle, and larger branched molecules in Fig. 3. Inspect `geometry.sdf`, `atom_coord.svg`, and `bond_coord.svg` to validate the final reconstruction.
- `4-1` through `4-3` correspond to the Fig. 4 resolution conditions. Compare the sharpness and topology of the atom/bond maps; these entries do not request graph construction.

To run one planar example only, execute the following from `<repo>/scripts`:

```
python ../src/test_ABNet_base.py \
--savedir ../examples/3-1_plane_mole/ \
--input_file ../examples/3-1_plane_mole/ters_map.pt \
--out_channels_a 4 --x_channels_a 64 64 96 --q_channels_a 64 64 64 \
--attention_type_a softmax --model_dir_a ../checkpoints/ANet_base/model_100.pth \
--out_channels_b 9 --x_channels_b 64 64 96 --q_channels_b 64 64 64 \
--attention_type_b sigmoid --model_dir_b ../checkpoints/BNet_base/model_100.pth \
--geom
```

`--geom` enables peak detection, connectivity inference, and SDF output. Without it, the program still produces the TERS, atom, and bond TIFF files but does not write coordinates or geometry files.

Step 2: Reproduce transfer-learning results for nonplanar molecules (Fig. 5)

```
cd <repo>/scripts
bash test_ABNet_nonplane_mole.sh
```

This script processes 5-1 through 5-4 with `test_ABNet_transfer.py` and enables `--geom`. Check the following in order:

1. `atom_map.tif`: whether C/N/O/H predictions remain spatially coherent;
2. `bond_map.tif`: whether the 11-channel bond/group prediction is well localized;
3. `geometry.sdf` and `bond_coord.svg`: whether the main molecular framework and substituent connectivity are chemically plausible.

The manuscript notes an important scope boundary: for strongly three-dimensional molecules, orientation-dependent selection rules and localized-field constraints can make the TERS information incomplete. Predictions outside the transfer-learning distribution should therefore be treated as candidate structures and chemical constraints, not as a guaranteed unique complete structure. Supplementary Fig. 7 illustrates this limitation.

Step 3: Reproduce simulated and experimental MgP results (Fig. 6)

```
cd <repo>/scripts
bash test_ABNet_plane_MgP.sh
```

The script separately processes simulated and experimental magnesium porphyrin (MgP). Compare `atom_map.tif` and `bond_map.tif` in `6-1_MgP_sim/` and `6-2_MgP_exp/` with Fig. 6a-c and Fig. 6d-f, respectively.

The experimental MgP maps originally have 25×25 pixels over the same 2.5×2.5 nm² area. The manuscript describes denoising and linear interpolation to 128×128 before network inference. The model recovers useful partial atom and connectivity constraints, but it does not fully reconstruct MgP at true atomic resolution: magnesium is not an output atom class, and experimental noise, spatial mismatch, and conjugated-state delocalization limit structural recovery. Interpret this example primarily through its probability maps rather than as a full SDF reconstruction benchmark.

Step 4: Train or retrain the models

```
cd <repo>
bash scripts/train_ABNet.sh
```

The training script trains base ANet/BNet and then initializes transfer learning from the base checkpoints. Its supplied settings are batch size 32, 100 epochs, Adam optimization, initial learning rate $3e-4$, decay factor 0.92 every 10 epochs, and channel-weighted MSE loss.

Important: the current repository does not include the complete `.npz` training arrays or index files referenced by the training script, such as `QM_FG_CN005A_H05_08A_mapping_data.npy` and `indices_QM5678_FG5678_x4.pth`. The release can reproduce its supplied inference examples, but it cannot be trained from scratch until those data files are supplied. Once available, place them at the referenced locations or explicitly update `--data_file`, `--targets_file`, and `--indices_file` (and the transfer-learning `*file1/*file2` paths).

Step 5: Validate the generated results

For an example run with `--geom`, validate the results in this order:

1. Open `ters_map_same_scale.tif`; ensure the 160 inputs are neither blank nor misoriented.
2. Open `atom_map.tif` and `bond_map.tif`; check for obvious global shifts, rotation, or mirror inconsistencies.
3. Confirm that `atom_coord.txt` and `bond_coord.txt` are nonempty and agree with their SVG visualizations.
4. Open `geometry.sdf` in a molecular viewer with V3000 SDF support; inspect connectivity and valence plausibility.
5. When comparing against manuscript/SI ground truth, distinguish the three processing layers: network probability maps, peak-detection output, and final graph construction.

5. Troubleshooting and Notes

The script cannot find `../src/...` or checkpoints

This is usually a working-directory error. Enter `<repo>/scripts` before running a Bash entry point, or rewrite every script path relative to the repository root.

ModuleNotFoundError, an incompatible PyTorch environment, or failure to load a checkpoint

Install the versions pinned in `requirements.txt` and keep `torch`, `torchvision`, and `torchaudio` mutually compatible. Model-construction arguments must also match the checkpoint: `x_channels`, `q_channels`, attention type, and output dimensions cannot be changed when loading the released weights.

Results are overwritten

Every inference run writes into `--savedir`. Create a dedicated output directory for new data or parameter experiments instead of reusing the supplied example directories.

TIFF files are generated, but no `geometry.sdf` exists

Check whether the command includes `--geom`. The preconfigured 4- * planar-resolution and MgP cases intentionally omit this switch.

Extra atoms, blurred bonds, or incomplete structures from experimental/low-resolution data

This can reflect an input outside the training distribution rather than an implementation failure. The SI resolution test uses a training baseline near 0.87 nm field confinement; at lower confinement around 1.12 nm, atom and

bond maps become diffuse and may contain extra predictions. Consider denoising, resolution/scale correction, or transfer learning with data from the target instrument. Retain and inspect probability maps and coordinates rather than relying only on one SDF structure.

Processing a new experimental TERS mapping

Match the training representation as closely as possible: identical spatial field of view, 128 x 128 grid, 160 integrated 20 cm⁻¹ channels, and compatible normalization/orientation. Start with one sample and inspect its input montage and probability maps before batch processing. For untrained elements, strongly delocalized conjugated systems, or pronounced 3D conformations, use the output as a structural hypothesis and chemical constraint, not an unconditional unique answer.

6. Recommended Reading and Execution Order

1. Read manuscript Fig. 1 and Supplementary Fig. 3 to understand the full sequence from TERS input to molecular graph.
2. Run `1_plane_mole` and inspect its TIFF, TXT, SVG, and SDF outputs one by one.
3. Run the Fig. 3 and Fig. 5 batches to compare planar and transfer-learning behavior.
4. Run MgP last, using Fig. 6 to understand experimental-domain limitations.
5. Attempt retraining only after obtaining the complete training arrays and index files.

7. License and Citation

The source code is released under the MIT License. Pretrained weights and benchmark datasets are released under CC-BY-NC 4.0. If you use TERS-ABNet, cite the associated paper: <<https://arxiv.org/abs/2603.21579>>.